Assignment 9

This exercise has two parts:

- Part A consists of mandatory exercises; you need to complete these and hand in your answers to be able to partake in the exam. This part also provides you with an impression of the minimum knowledge level necessary to pass the exam.
- Part B consists of additional exercises. These repeat a concept that is covered in Part A or help to broaden or deepen your understanding of some of the topics.

Assignment by

Lan Stare, s1169977

Aaron Ritz, s1170346

Advised Reading and Exercise Material

The following reading material is recommended:

Read and study chapter 5 of TSK. You can skip section 5.2.6.

Additional Tools

For this exercise you will need to load the provided *clusterPlot.py* script from the toolbox folder.

Introduction

In previous exercises we considered supervised learning, i.e., we were given both input data **X** and output values **Y**. We now move on to unsupervised learning where we are only provided input data **X**. The aim is here to find common patterns in the data such as groups of observations that are similar in some sense.

Part A - Mandatory Exercises

Below you will find the mandatory exercises for Week 4 of the course Data Mining and Machine Learning.

Exercise 1 - *k*-means clustering

In this part of the exercise we will investigate *k*-means clustering. In *k*-means each of the data points are assigned to the cluster in closest proximity according to some measure of distance between cluster centers and data points. When the distance is given by the squared euclidian distance, the centers are also called centroids. Once the data points have been assigned, each cluster center is updated to be placed at the center of the data points that are assigned to the cluster. This continues iteratively, usually until the assignment of data points to centers no longer changes or until a maximal number of iterations is reached.

Exercise 1.1

Implement the K-means algorithm. Pseudo-code for this algorithm is given in section 5.2.1 of TSK, Algorithm 5.1. Use the standard (squared) Euclidean distance.

Helpful hints:

- You can sample k random datapoints from some dataset d using random.sample(d,k)

In [1]:

```
import numpy as np
import random

def squared_euclidean(x, y):
    return np.linalg.norm(x - y)

def initialize_centroids(data, k):
    idx = random.sample(range(len(data)), k)
    return [data[i] for i in idx]

def assign_cluster(data, centroids):
    labels = []   # cluster index for each point

    for point in data:
        # compute distances to each centroid
        distances = [squared_euclidean(point, c) for c in centroids]
        # pick the closest centroid index
        closest = int(np.argmin(distances))
        labels.append(closest)

    return labels


def compute_centroid(data, labels, k):
    new_centroids = []
    for c in range(k):
        pts = [data[i] for i in range(len(data)) if labels[i] == c]
        new_centroids.append(np.mean(pts, axis=0))
    return np.array(new_centroids)
```

In [2]:

```
# Write your code for 1.1 here, ideally use the helper functions
# return both your final cluster assignments, and the centroids!

def kmeans(data, k, tolerance=1e-4, max_iter=300):  #I added max_iter for safety although it usually converges quickly :)
    centroids = initialize_centroids(data, k)

    for _ in range(max_iter):
        labels = assign_cluster(data, centroids)
        new_centroids = compute_centroid(data, labels, k)

        movement = np.sum([
            squared_euclidean(centroids[i], new_centroids[i])
            for i in range(k)
        ])

        centroids = new_centroids

        if movement < tolerance:
            break

    return np.array(labels), centroids
```

Exercise 1.2

Load the `synth1` data into python using the `loadmat` function. The `synth1` data is a Matfile loaded into Python as a dictionary; the relevant data is contained under keys `X` and `y`. Cluster the data into `K = 4` clusters using the *k*-means algorithm. Make a scatter plot of the data and the clustering using the provided `clusterPlot()` function from the toolbox. Move the legend if the position is obfuscating data.

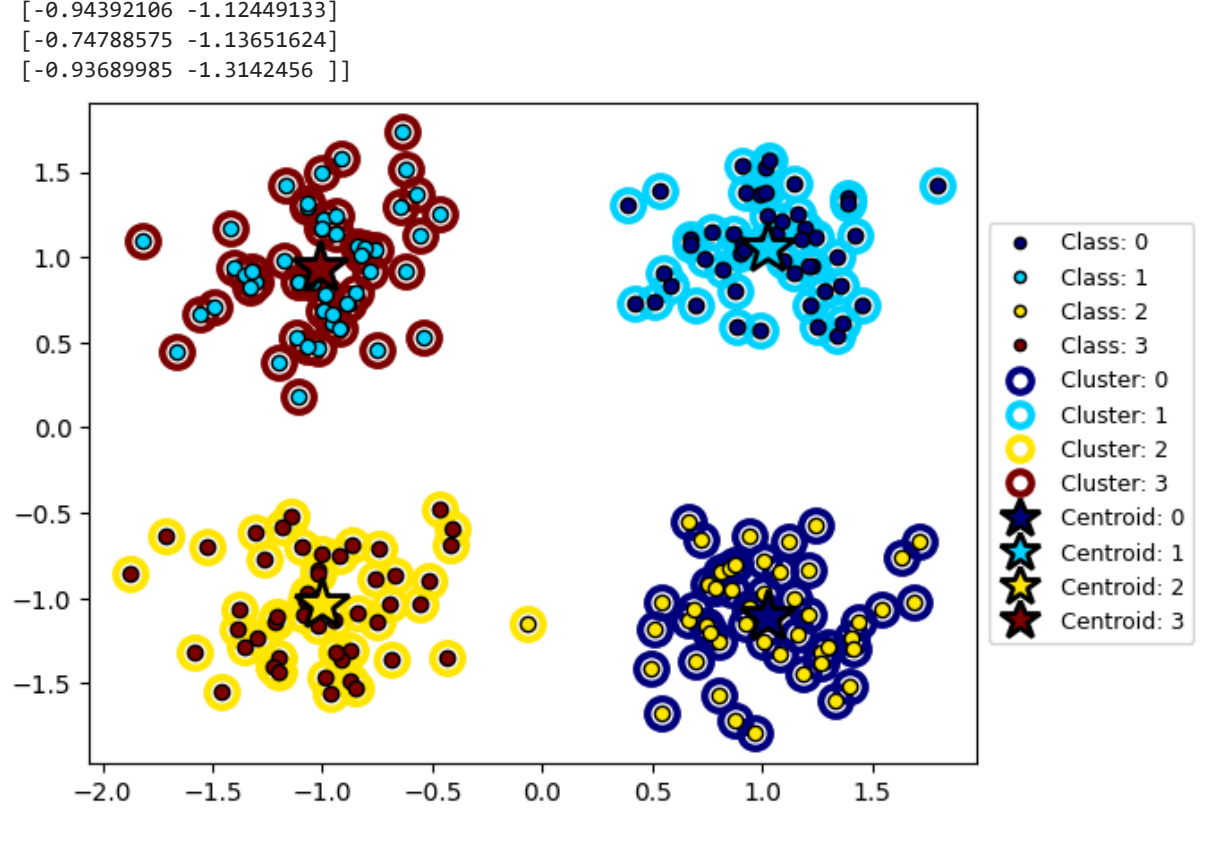
Helpful hints:

- Type `clusterPlot(X,clusters,centroids,y)` to plot the data and the clustering. Don't forget to import the function from the `toolbox` folder. If you are unsure about how to use the function, open the .py file in a text editor and take a look at documentation.
- Make sure that your code from 1.1 returns the clusters and centroids properly!

In [ ]:

```
## Answer to exercise 1.2

import scipy.io as si
from toolbox import clusterPlot as cp

X = si.loadmat('data/synth1.mat')['X']     #n x 2 matrix of points
y = si.loadmat('data/synth1.mat')['y']     #true class labels
#print(X)
#print(y)

#plotting the clusterPlot:
clusters, centroids = kmeans(X, 4)
cp.clusterPlot(X, clusters, centroids, y)
```

```
[[ 0.41819347  0.72505978]
 [ 1.16077592  1.25372183]
 [ 0.99195122  0.57332584]
 [ 1.05911131  1.14392175]
 [ 0.86661635  1.14353708]
 [ 1.01605275  1.52383058]
 [ 0.99381613  1.36534343]
 [ 1.14670371  1.43107459]
 [ 0.90348822  1.01994253]
 [ 1.14103296  0.91031573]
 [ 1.22114579  0.71690785]
 [ 0.38908409  1.30626919]
 [ 1.02521334  1.24365688]
 [ 0.82213277  0.92948993]
 [ 1.02862273  1.56489411]
 [ 1.19196182  1.16853147]
 [ 0.88276935  0.59449177]
 [ 1.28061581  0.79919027]
 [ 1.4206002   1.12939987]
 [ 1.33783214  0.99801456]
 [ 0.51167748  0.7437484 ]
 [ 0.53556441  1.38879855]
 [ 1.24951175  0.58850582]
 [ 0.91006184  1.07815047]
 [ 1.33773105  0.53949808]
 [ 1.36182023  0.61468247]
 [ 0.58067327  0.83714056]
 [ 1.01520555  1.10036993]
 [ 1.24044894  1.11473397]
 [ 1.0960145   0.98346027]
 [ 1.01175877  1.38436726]
 [ 1.38585908  1.35374649]
 [ 0.90584544  1.53428867]
 [ 0.92207457  1.37561863]
 [ 1.39016089  1.31454408]
 [ 0.91283882  1.04737244]
 [ 0.87510962  0.80762173]
 [ 0.67263138  1.11078808]
 [-0.61727309  0.91447581]
 [-0.8422087   1.06518992]
 [-1.40210832  0.93931892]
 [-1.04297797  0.85975734]
 [-1.06961091  0.86004266]
 [-1.10938804  0.17848508]
 [-1.41958609  1.16715508]
 [-0.46467312  1.25301782]
 [-0.99646857  1.22227124]
 [-1.35699085  0.90122905]
 [-1.02286521  0.46382468]
 [-1.66457848  0.44684486]
 [-0.75569289  1.04973794]
 [-1.1209121   0.53073121]
 [-0.93788085  1.13462457]
 [-0.75344319  0.46134017]
 [-0.80876595  1.05610234]
 [-0.61880319  1.51648519]
 [-0.56970124  1.37122439]
 [-0.9980607   0.68899031]
 [-0.91594674  1.58215347]
 [-1.30749739  0.85399981]
 [-0.95779575  0.61185017]
 [-1.33087715  0.82098712]
 [-1.00157939  1.16628699]
 [-1.19552166  0.38274996]
 [-1.16915221  1.41845859]
 [-1.00203535  1.50093254]
 [-1.81706128  1.09286323]
 [-0.64798366  1.29426782]
 [-1.48980272  0.70821751]
 [-0.92043747  0.58662299]
 [-0.55629556  1.12429951]
 [-1.06841311  0.47851441]
 [-1.3199018   0.92367922]
 [-0.88364283  0.72721768]
 [-0.82471051  1.01168892]
 [-1.06502482  1.29396547]
 [-1.5568985   0.66534582]
 [-1.11167992  0.85598285]
 [-0.95782733  0.66885188]
 [ 1.43362921 -1.135497  ]
 [ 0.87932258 -1.71671488]
 [ 0.69377287 -1.36604624]
 [ 0.92494485 -1.14880758]
 [ 1.40380738 -1.23680733]
 [-0.06440115 -1.14840825]
 [ 1.00779196 -0.97465342]
 [ 1.16170165 -1.21138408]
 [ 0.9434351  -0.64065772]
 [ 0.49335343 -1.41419084]
 [ 1.07904522 -0.84662225]
 [ 1.0092399  -0.7838784 ]
 [ 1.68565151 -1.02197386]
 [ 0.54051113 -1.02354189]
 [ 1.21034254 -0.84000468]
 [ 0.6614522  -0.54849148]
 [ 0.74999055 -0.92337646]
 [ 0.74306477 -1.15992523]
 [ 1.14274154 -1.00755037]
 [ 0.85931529 -0.94672848]
 [ 1.53958568 -1.06217537]
 [ 0.81476014 -0.84251478]
 [ 1.00665764 -1.25901124]
 [ 0.96313959 -1.79571576]
 [ 0.8064161  -1.25381891]
 [ 1.01719409 -1.07371806]
 [ 1.41555099 -1.29896904]
 [ 1.24334084 -0.5674698 ]
 [ 1.08076687 -1.32694606]
 [ 1.26936349 -1.31605106]
 [ 1.39421182 -1.52111769]
 [ 1.32906754 -1.60397275]
 [ 0.50694755 -1.18275183]
 [ 1.71581352 -0.66198238]
 [ 0.75829104 -1.20563561]
 [ 0.53931623 -1.671931  ]
 [ 0.85039278 -0.82048818]
 [ 1.62914002 -0.76116729]
 [ 0.80245206 -1.568653  ]
 [ 0.94084181 -1.05447247]
 [ 0.78568567 -0.94211125]
 [-0.95888132 -1.55993495]
 [-1.45721363 -1.55113147]
 [-1.52452982 -0.70364555]
 [-1.21466781 -1.13974244]
 [-0.87710446 -1.31077431]
 [-1.70806791 -0.63320323]
 [-0.69488691 -1.03966167]
 [-1.09600423 -0.70287   ]
 [-0.91805793 -1.3631952 ]
 [-1.06958197 -0.96736913]
 [-0.872032   -1.48292582]
 [-0.6663459  -0.8634077 ]
 [-1.30460771 -0.60959   ]
 [-0.99121155 -1.46601622]
 [-1.00098405 -0.7431341 ]
 [-0.55988568 -1.03602192]
 [-1.01598632 -0.83194113]
 [-1.01821708 -0.85633079]
 [-0.68509831 -1.3551633 ]
 [-1.29641983 -1.23723174]
 [-1.38303629 -1.1795339 ]
 [-1.14297404 -0.51766252]
 [-1.18433487 -0.58111282]
 [-1.35188004 -1.28803104]
 [-1.20989558 -1.10982221]
 [-1.02163838 -1.16523973]
 [-1.57727018 -1.31495561]
 [-1.19481926 -1.34744633]
 [-0.83946024 -1.08330877]
 [-1.37506809 -1.06508378]
 [-0.41372666 -0.69019744]
 [-0.5162402  -0.90234885]
 [-0.43689812 -1.35329466]
 [-0.74388397 -0.70932358]
 [-1.08026055 -1.01328453]
 [-0.84574141 -1.53150518]
 [-1.26421015 -0.7689658 ]
 [-1.87491398 -0.85808428]
 [-1.22425213 -1.40622559]
 [-1.08690083 -1.09944708]
 [ 1.22293391  0.9546126 ]
 [ 1.17565517  1.10601265]
 [ 0.69580575  0.7219573 ]
 [ 0.73910372  0.99716201]
 [ 1.45745021  0.71784122]
 [ 1.35248177  0.83176738]
 [ 0.67285068  1.07799117]
 [ 1.09062482  1.21549856]
 [ 0.54697183  0.91263228]
 [ 1.20687391  0.95371421]
 [ 1.79363967  1.42314306]
 [ 0.77237366  1.14446767]
 [-0.53891523  0.52557509]
 [-0.785871    0.92271531]
 [-1.17690773  0.97814939]
 [-0.85161159  0.78797904]
 [-1.06646209  1.31290247]
 [-0.63924184  1.73256484]
 [-0.93526693  1.24013954]
 [-0.98531246  0.77891718]
 [-0.88862435  0.73339048]
 [ 0.72357884 -0.65624055]
 [ 0.66507275 -1.12898017]
 [ 0.87838015 -0.80811987]
 [ 1.11983142 -0.67047488]
 [ 1.18609431 -1.4436833 ]
 [ 1.21341444 -1.0947946 ]
 [ 1.26868273 -1.37996386]
 [ 1.30219719 -1.28535675]
 [ 0.69236598 -1.0677577 ]
 [-0.40875962 -0.59880061]
 [-0.86453062 -0.69097668]
 [-0.92410205 -0.75160645]
 [-0.46430938 -0.47742314]
 [-1.04437844 -1.01351879]
 [-1.19948078 -1.42933782]
 [-0.75629165 -0.89092626]
 [-0.94392106 -1.12449133]
 [-0.74788575 -1.13651624]
 [-0.93689985 -1.3142456 ]]
```



Exercise 1.3

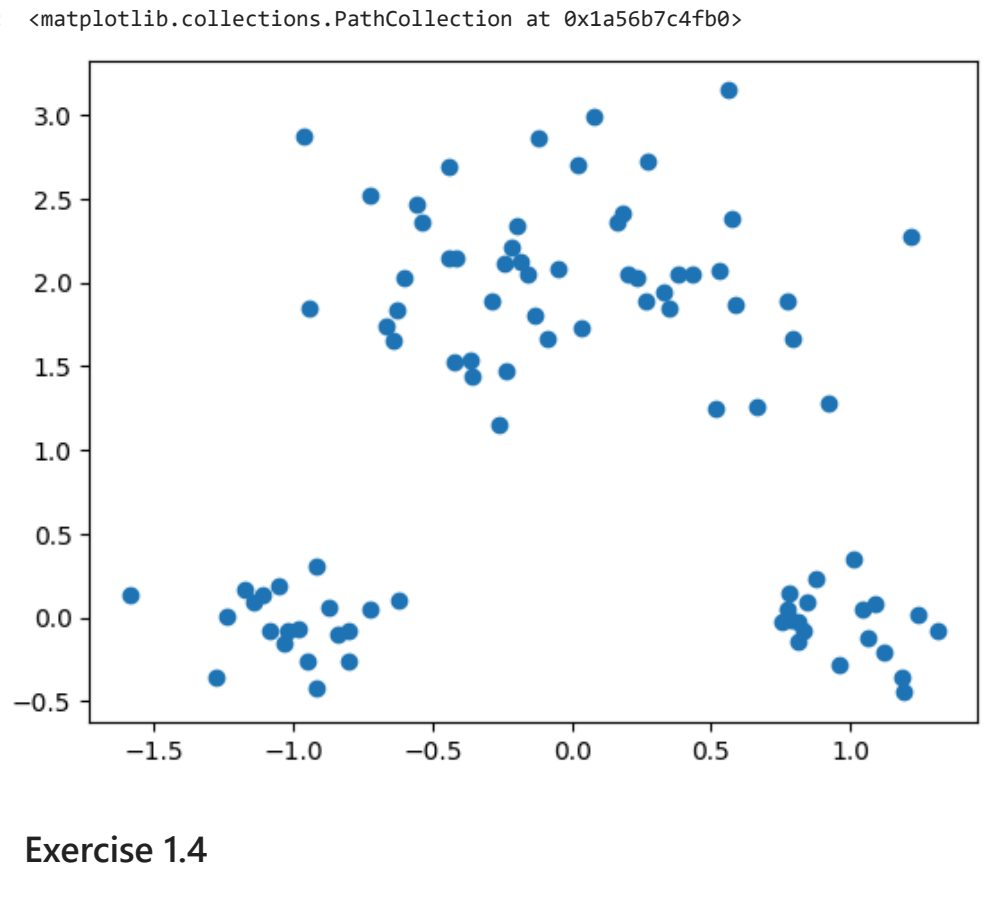
Download the dataset from http://www.cs.ru.nl/~tomh/onderwijs/dm/dm_files/dataforkmeans.csv. Plot a scatterplot to inspect the data and see what it looks like.

In [4]:

```
## Answer to exercise 1.3
import pandas as pd
import matplotlib.pyplot as plt

data2 = pd.read_csv("dataforkmeans.csv")
x21, x22 = data2.iloc[:, 0], data2.iloc[:, 1]
plt.scatter(x21, x22)
```

Out[4]:

```
<matplotlib.collections.PathCollection at 0x1a56b7c4fb0>
```

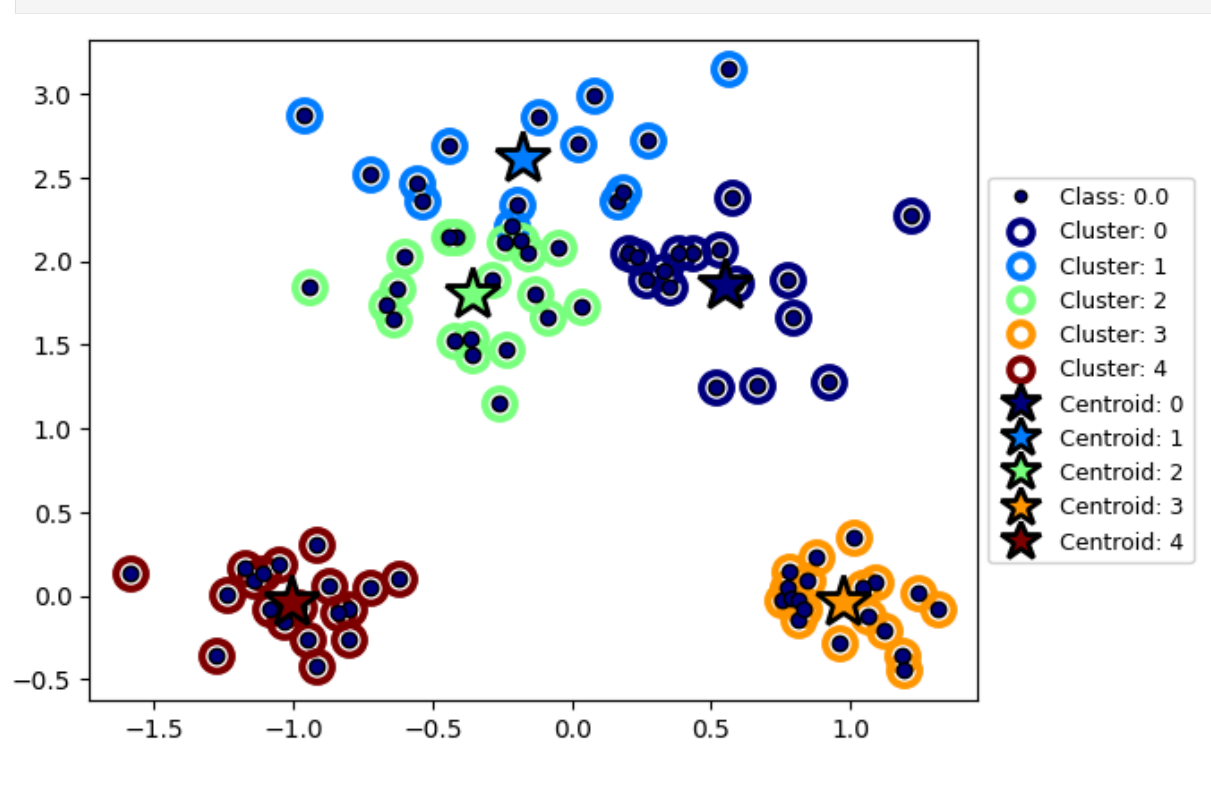


Exercise 1.4

Apply your K-means algorithm from 1.1 to the dataset from 1.3. Play around with different values of K (the number of clusters) and plot a typical result for K = 5.

In [6]:

```
## Answer to exercise 1.4
x3 = np.array(list(zip(data2.iloc[:, 0], data2.iloc[:, 1])))

clusters5, centroids5 = kmeans(x3, 5)
cp.clusterPlot(x3, clusters5, centroids5)
```



Exercise 1.5

Run the K-means algorithm from different initializations for K = 3.

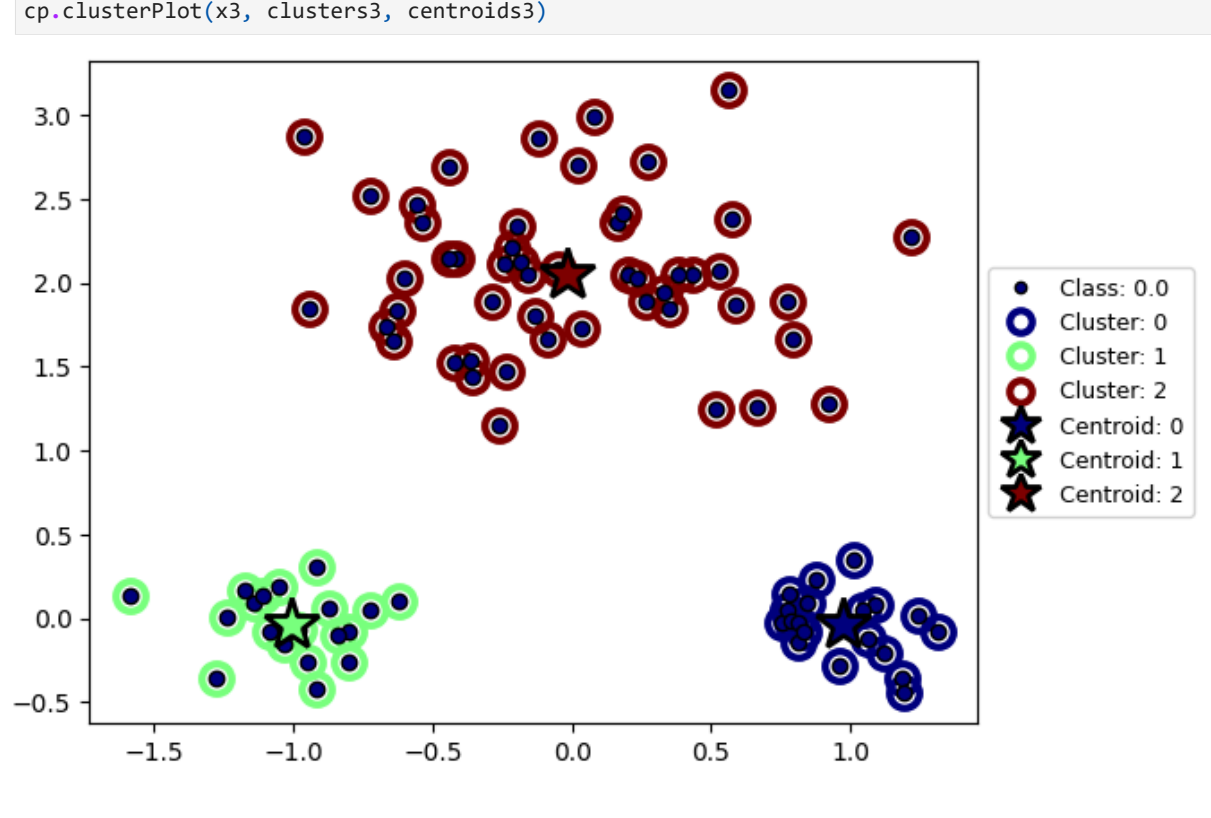
How many different solutions do you get? What are the typical solutions when you categorize these solutions? Either sketch or describe these.

Helpful hints:

- initialize the means to 3 different randomly chosen data points
- You will (hopefully) find that many of these solutions are somewhat similar.

In [7]:

```
## Answer to exercise 1.5
clusters3, centroids3 = kmeans(x3, 3)
cp.clusterPlot(x3, clusters3, centroids3)
```

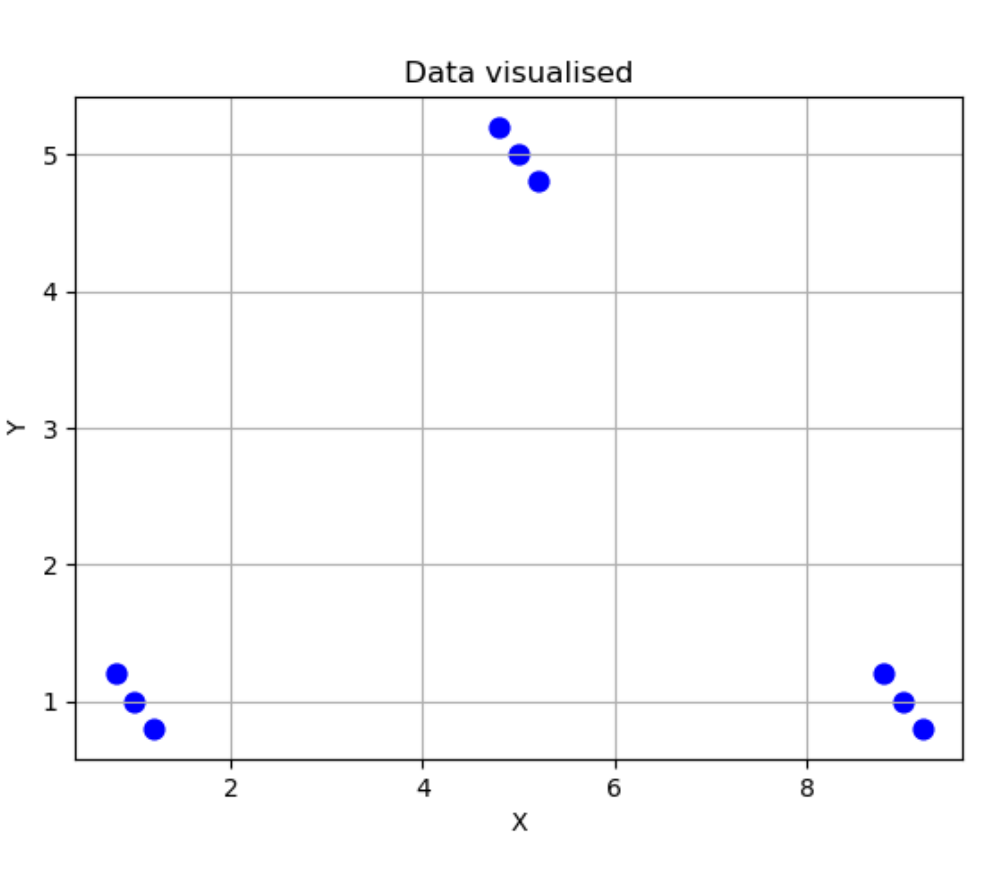


ANSWER: You mostly get 2 very different solutions. The first solution you get is when one centroid is at approximately (-1.0, 0.0), another one at approx. (1.0, 0.0) and the third one at approx. (0.0, 2.0). The second solution you get if when one centroid is at approx. (0.0, 0.0), another one at approx. (-0.5, 2.0) and the third one at (0.5, 2.0). There are about 2 more solutions that you get but are very similar to the second one.

Exercise 2 - *k*-means initialization

The way our original centroids are selected can have a massive impact on the final performance of k-means. In the table below we provide a set of points, this set is visualised in the image in the cell below as well. Clearly these points form 3 distinct clusters, right?

Point	X	Y
A_1	1.0	1.0
A_2	1.2	0.8
A_3	0.8	1.2
B_1	5.0	5.0
B_2	5.2	4.8
B_3	4.8	5.2
C_1	9.0	1.0
C_2	9.2	0.8
C_3	8.8	1.2



Exercise 2.1

If you take your initial centroids to be A_1, B_1 and C_1, briefly describe what you expect to happen after k-means clustering.

ANSWER: After k-means clustering, the final centroids would not change. The algorithm would run for exactly 1 iteration and then stop, since the centroids wouldldn't change.

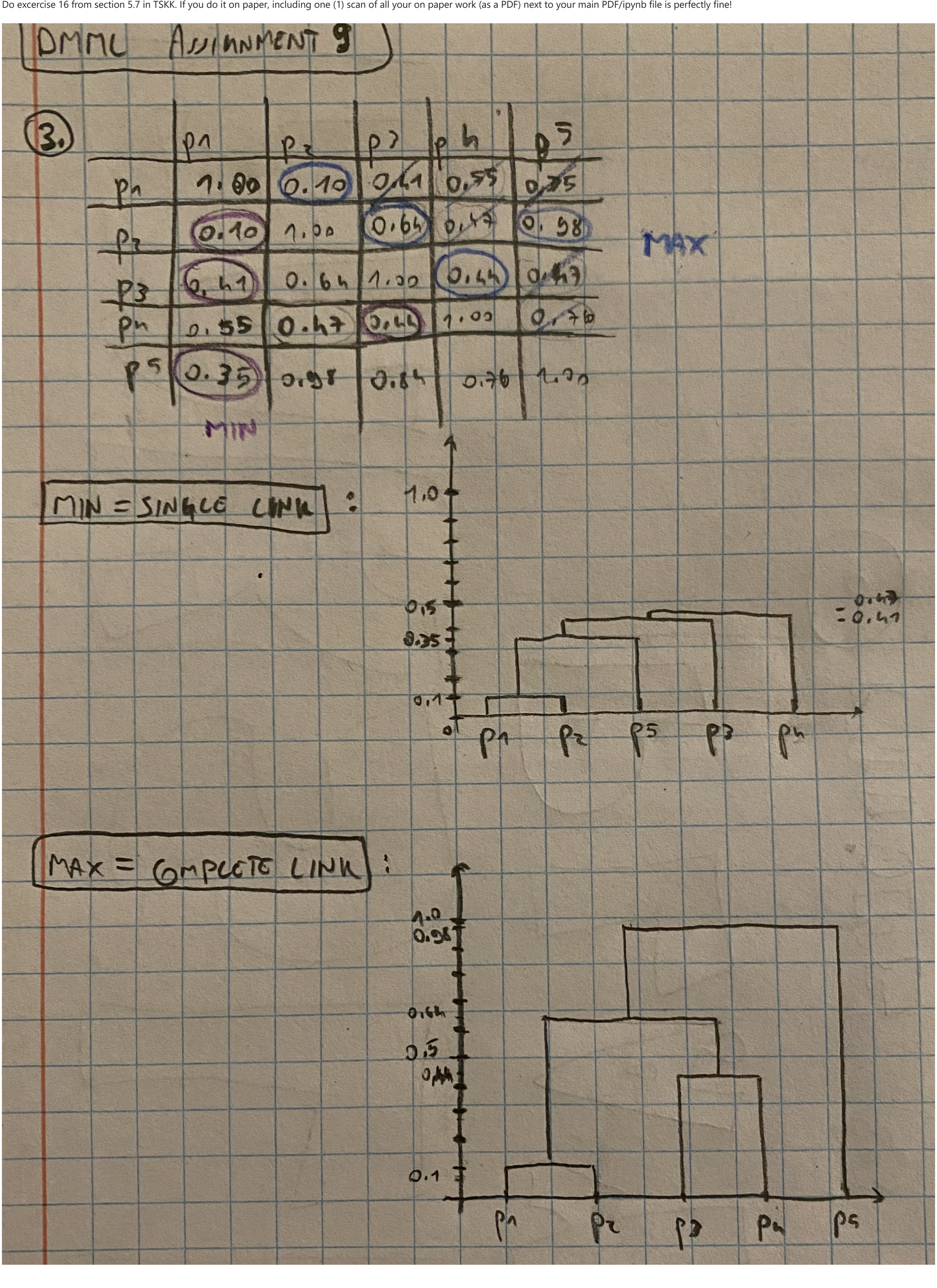
Exercise 2.2

Is there any initialization with $k = 3$ for which you see undesired behaviour? Give an example of such an initialization and explain why this would go wrong.

ANSWER: Yes, if we choose A1, A2 and B1 as initial centroids, we would get undesired behaviour. Since the distance between B'a and C's is shorter, C1, C2 and C3 would all be asigned to B1 in the first iteration. That would mean that the centroid B2 would move in between B1, C1, C2 and C3, whereas A3 would be assigned to A1, so the centroid A1 would shift in between A1 and A3. A2 wouldn't budge. The centroids wouldn't change after that, since all C's would still be assigned to the centroid that was once B1.

Exercise 3 - On-paper hierarchical clustering

Do excercise 16 from section 5.7 in TSKK. If you do it on paper, including one (1) scan of all your on paper work (as a PDF) next to your main PDF/ipynb file is perfectly fine!



DMML ASSIGNMENT 9

3.

	p1	p2	p3	p4	p5
p1	1.00	0.10	0.41	0.55	0.35
p2	0.10	1.00	0.64	0.47	0.98
p3	0.41	0.64	1.00	0.44	0.47
p4	0.55	0.47	0.44	1.00	0.76
p5	0.35	0.98	0.84	0.76	1.00

MAX

MIN

MIN = SINGLE LINK :

MAX = COMPLETE LINK :

Exercise 4 - Hierarchical Clustering

In this part of the exercie we will consider hierarchical clustering based on the functions from the package `scipy.cluster.hierarchy`. The function `linkage` forms a sample to sample distance matrix according to a given distance metric, and creates the linkages between data points forming the hierarchical cluster tree. The function *dendrogram* creates a plot of the generated tree. The function `fcluster` extracts the cluster from a linkage matrix w.r.t. a given criterion. Use *help* for the three functions (or read the online documentation) and inspect what distance metrics and linkage functions are implemented.

Exercise 4.1

Load the data set from `data/synth1`. Cluster the data using hierarchical clustering with single linkage using the Euclidean distance measure. Cluster the data into 4 clusters by cutting off the dendrogram at a threshold. Plot a dendrogram and a scatter plot of the clusters.

- The function `linkage` computes the hierarchical clustering, resulting in a matrix representing the hierarchy of clusterings. TRead the online documentation to learn how to use it.
- You can e.g. type `Z = linkage(X, method='single', metric='euclidean')` to use single linkage with the Euclidean distance measure.
- To compute a clustering, you can use the function `fcluster`. For example, type `cls = fcluster(Z, criterion='maxclust', t=4)` to get a maximum of 4 clusters. Type `help(fcluster)` (or read the online documentation) to learn more about what this function does.
- To plot a dendrogram, you can use the `dendrogram` function. You can make the visualization prettier by choosing an appropriate `color_threshold` to visually obtain 4 cluster. Note that this change in visualization does not impact the actual amount of clusters obtained by using the `fcluster` function.
- Again, you can use the function `clusterPlot` to plot a scatter plot of the clustering.

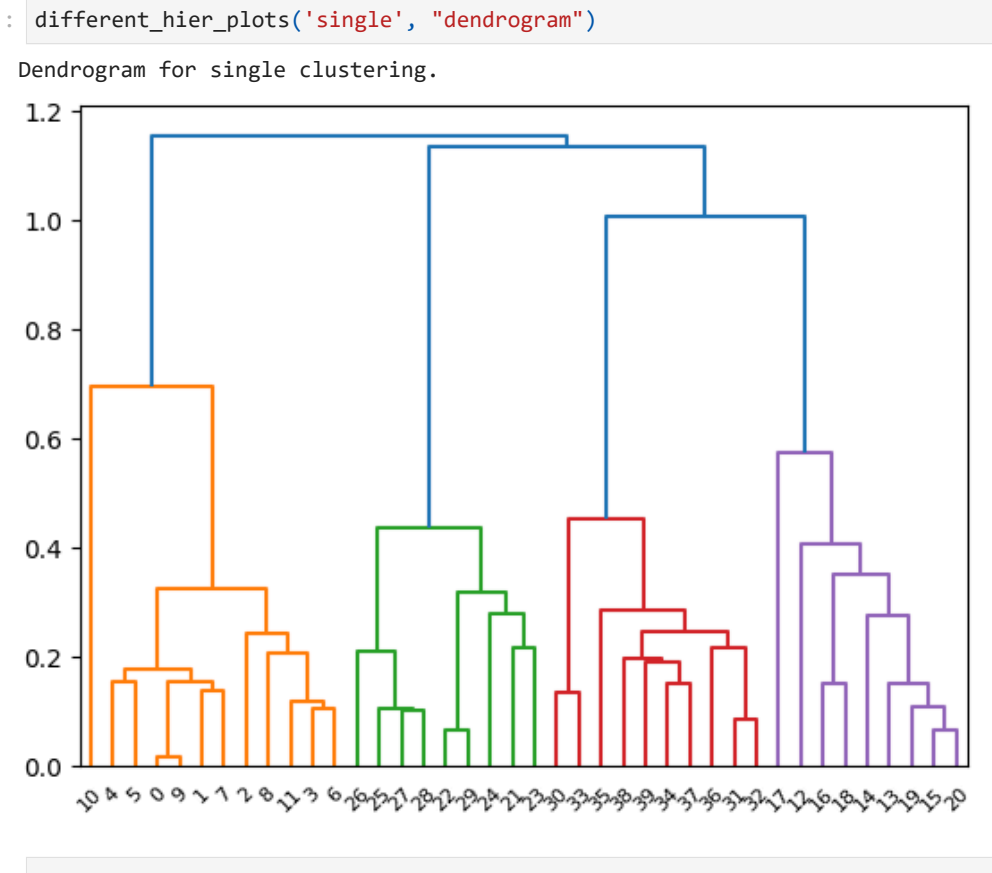
In [32]:

```
##Answer to question 4.1

import scipy.io as si
import scipy.cluster.hierarchy as sch

#reading the data:
data = si.loadmat('data/synth1.mat')['X_test']

#hirarchical clustering in a matrix representation:
def different_hier_plots(t, typ):
    clustering = sch.linkage(data, method=t, metric='euclidean')

    #computing the clustering with 4 clusters:
    cls = sch.fcluster(clustering, criterion='maxclust', t=4)

    #plotting the dendrogram:
    if typ == "dendrogram":
        print(f"Dendrogram for {t} clustering.")
        sch.dendrogram(clustering)

    #plotting the scatterplot:
    if typ == "scatterplot":
        print(f"Scatterplot for {t} clustering.")
        cp.clusterPlot(data, cls)
```

In [33]:

```
different_hier_plots('single', "dendrogram")
```

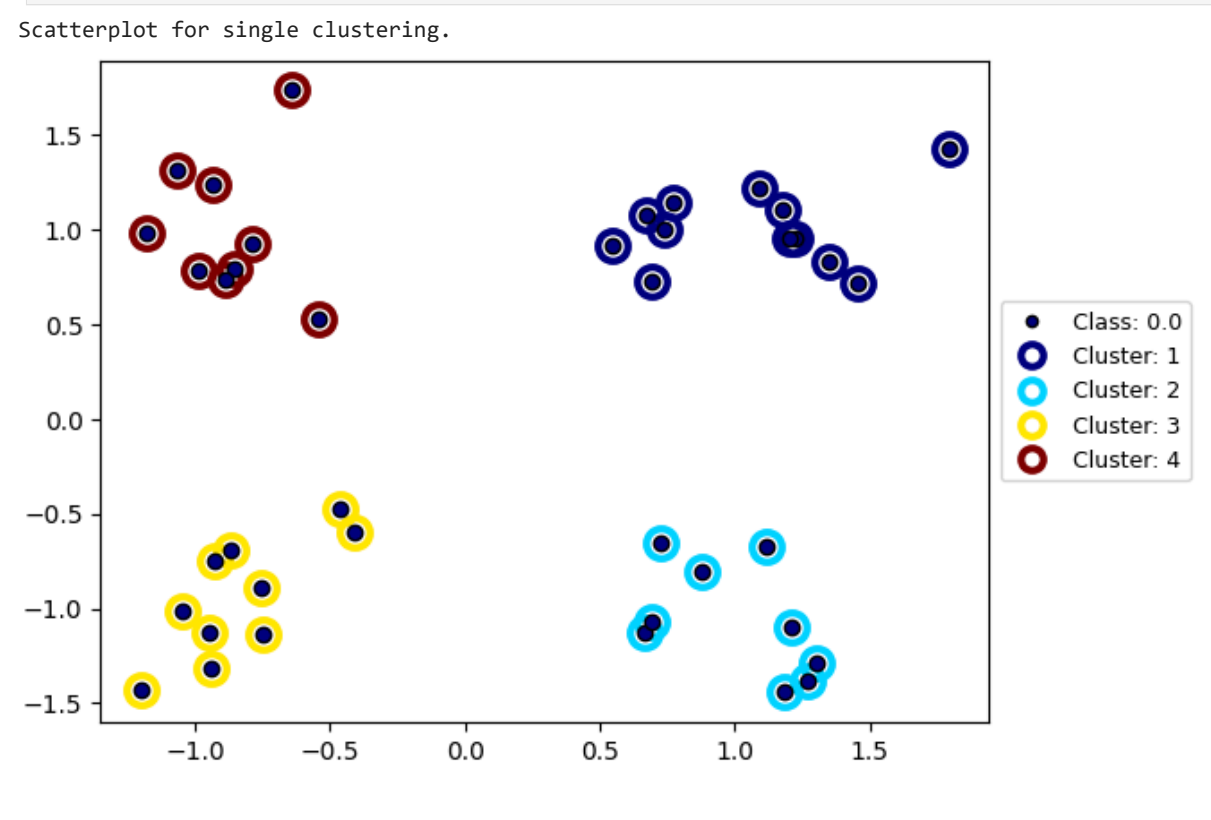
```
Dendrogram for single clustering.
```



In [23]:

```
different_hier_plots('single', "scatterplot")
```

```
Scatterplot for single clustering.
```



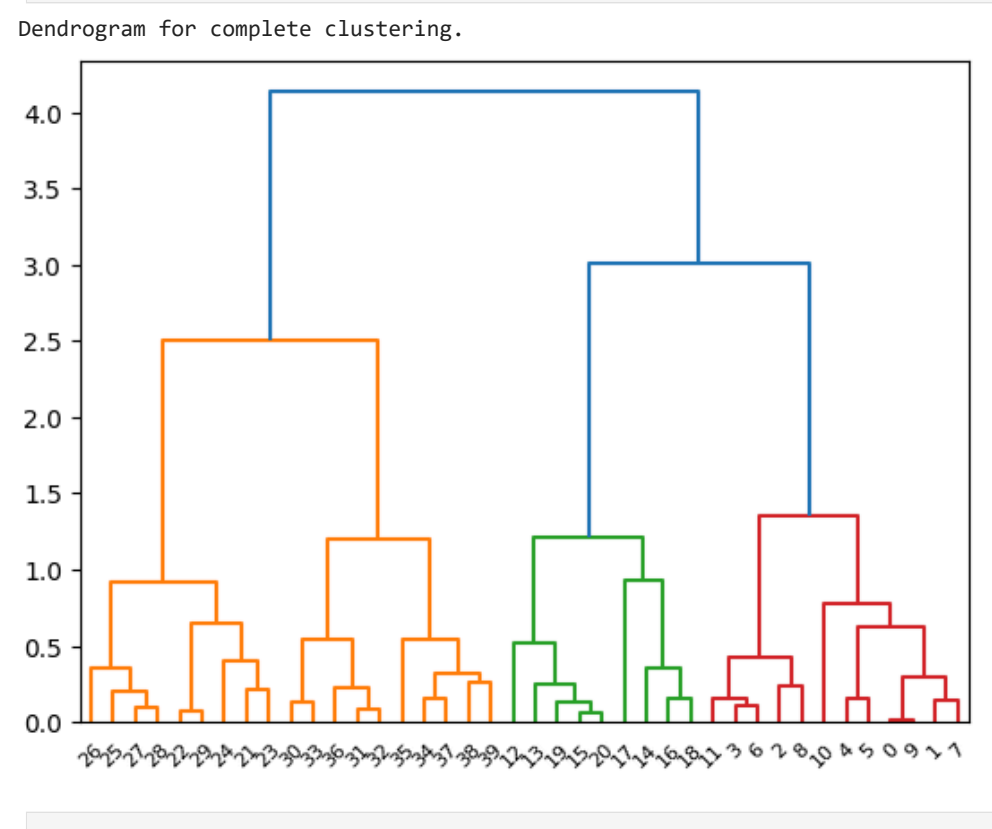
Exercise 4.2

Change the linkage methods (single, complete, average) for the `synth1` dataset and **explain** how this changes the dendrogram (plot all 3 dendrograms) and make a scatterplot of the best performing method. Argue why you think this linkage method performs best. If several linkage methods perform similarly, describe why. Repeat this exercise for the `synth2`, `synth3`, and `synth4` datasets.

In [28]:

```
different_hier_plots('complete', "dendrogram")
```

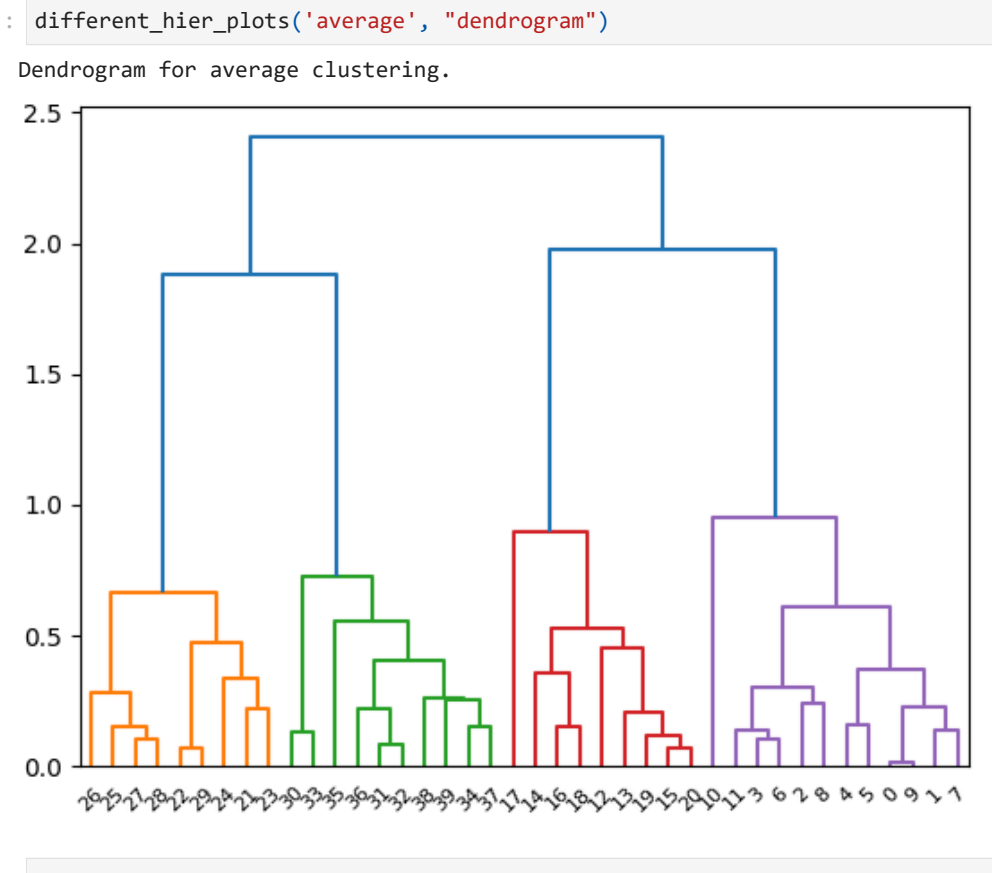
```
Dendrogram for complete clustering.
```



In [27]:

```
different_hier_plots('average', "dendrogram")
```

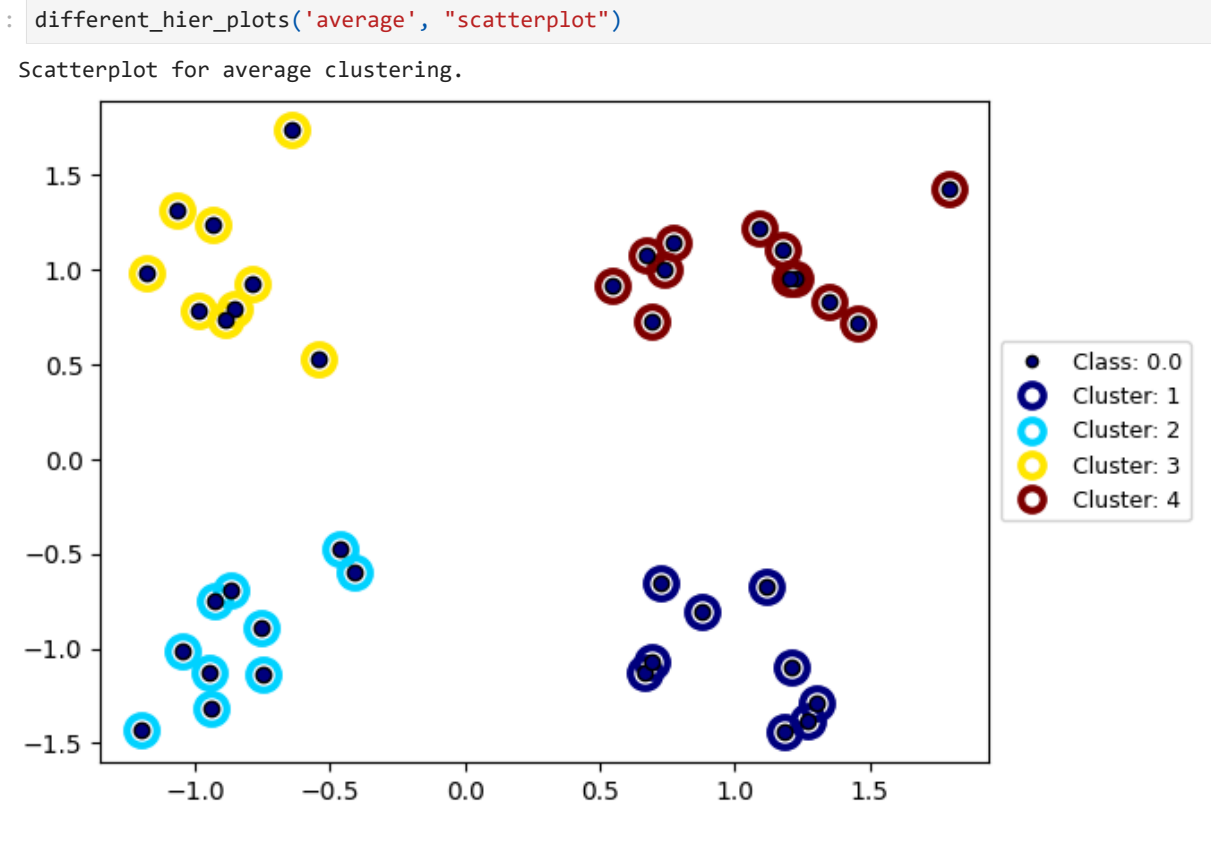
```
Dendrogram for average clustering.
```



In [36]:

```
different_hier_plots('average', "scatterplot")
```

```
Scatterplot for average clustering.
```



ANSWER:

Average and complete methods both produce dendrograms that differentiate the clusters better, although all of them still correctly (equally) cluster the points, which is seen by equivalent scatterplots. For some reason, the complete method colors clusters 1 and 2 with the same color but still differentiates them if we plot the scatterplot. Because the ratio between points that are in the same cluster and points from different clusters is higher in the average method, it is preferred.

It's hard to explain how the dendrogram changes depending on the method. We can see that in the single method cluster 1 is consider the most different / distant from all the others while in the complete and average, clusters 1 and 2 are most different / distant from clusters 3 and 4. Another thing is that the single method has the lowest values of distances and the complete method has the highest values of distances.

Exercise 5

Read https://proceedings.neurips.cc/paper_files/paper/2002/file/43e4e6a6f341e00671e123714de019a8-Paper.pdf and answer the following questions:

1. Briefly explain what scale-invariance, richness and consistency each mean.
2. Why do we want all 3?
3. Briefly and informally: why can't all 3 hold at the same time?
4. What impact does it have that not all 3 can hold at the same time? (Think about why different methods give different results)

ANSWER:

1. Scale-invariance: Clustering should not change if all distances are multiplied by the same positive constant; changing measurement units should not affect the outcome.

Richness: For every possible partition of the dataset, there exists some distance function for which the algorithm outputs exactly that partition.

Consistency: If the algorithm outputs a clustering for some metric, and we modify that metric by shrinking all within-cluster distances and expanding all between-cluster distances, then the algorithm should output the same clustering.

2. Scale-invariance avoids dependence on arbitrary units; richness ensures the algorithm can represent any reasonable clustering; consistency ensures stability when evidence for the clustering becomes stronger. Together they describe intuitive, desirable behavior for general-purpose clustering.

3. Richness allows any clustering to be the output for some metric; consistency forces the output to remain unchanged under certain metric transformations; scale-invariance prevents distinguishing metrics by scaling. These constraints conflict, so one axiom forces one output while another forces a different one, making it impossible to satisfy all three at the same time.

4. Every clustering algorithm must violate at least one axiom, which explains why different methods give different results. There is no universally correct clustering method; each makes trade-offs among flexibility, stability, and scale behavior. In practice, we choose the method based on which property is most important for the task.

Part B

Exercise B.1 - Practical applications of k-means

k-means clustering has many different applications, one of which is data compression. A data set can be compressed by performing *k*-means clustering and then representing each data object by the centroid of the corresponding cluster. Thus, the only data that need to be stored are the *K* cluster centers and the *N* cluster indices.

Exercise B.1.1

We will take another look at the MNIST digits data. Load the data, `data/digits.mat` using the `loadmat` function. Each data object is a $16 \times 16 = 256$ dimensional vector, corresponding to a grayscale 16×16 pixels image. Compute a *k*-means clustering of the data with *K = 10* clusters. Plot a few random images from the data set as well as their corresponding cluster centroids to see how they are represented.

Helpful hint:

- You can plot an image by the command `imshow(np.reshape(X[k,:],(x,y)))` which reshapes an image vector to a 2-dimensional array and plots it. You can plot the cluster centroids in a similar manner.
- If you can not figure out how to get 1.1 working, you can use the function `KMeans()` from the package sklearn.cluster to compute *k*-means clustering. please read the documentation of the function, as it creates an object which you can then use for clustering.

In [9]:

```
## Answer to exercise 2.1
raise NotImplementedError()
```

```
---------------------------------------------------------------------------
NotImplementedError                       Traceback (most recent call last)
Cell In[9], line 2
      1 ## Answer to exercise 2.1
----> 2 raise NotImplementedError()

NotImplementedError: 
```

Exercise B.1.2

How well is the data represented by the cluster centroids? Why does running *k*-means with *K = 10* not give you 10 clusters corresponding to the 10 digits *0..9*? How many clusters do you need to visually represent the 10 different digits? Are there any digits that the clustering algorithm seems to confuse more than others and can you explain why?

YOUR ANSWER

Exercise B.2

For further practice, e.g. to prepare for the exam: Do excercises 5,6,17, 20 in section 5.7 in TSKK.

Handing in

Hand in the filled-in notebook:

- Make sure your names and student-numbers are mentioned at the appropriate location at the top of this file.
- Make sure you wrote a serious attempt for all exercises in part A
- Make sure the notebook is named groupnumber_assignment9.ipynb

When you are confident all of this is correctly done, one of you can hand in the exercises in Brightspace!